

ФГБОУ ВПО КубГУ

Курс лекций

Объектно-ориентированное программирование

2025 г

снс ИММИ КубГУ, к.ф.-м.н, доцент
каф. ВМиИ КубГУ, Фоменко С.И.

Тема 2: ООП в C++

1. Объявление класса: синтаксис.
2. Конструкторы и деструкторы
3. Перегрузка методов, конструкторов и операций
4. Наследование
5. Связывание, виртуальные методы
6. Шаблоны и обобщённое программирование

Объявление класса (базового)

```
class Имя_Класса
{
   тело_класса
}
```

`тело_класса:`

1

```
Спецификатор_доступа :
private:   члены класса доступны только внутри этого класса
 (приватные члены класса) – по умолчанию.
protected: члены класса доступны только внутри этого класса и
 в потомках (защищенные члены класса)
public:    доступ к членам класса неограничен (открытые члены
 класса)
```

Объявление класса (базового)

```
class Имя_Класса1
{
    type1 atr1, atr2  // приватные атрибуты типа type1
    type  method1(type1 var1, type1 var2) // приватный метод
        {тело метода}

public: // публичные атрибуты/ методы

Имя_Класса1(type1 var1, var2) // конструктор
        {тело метода}
~Имя_Класса1(type1 var1, var2) // деструктор
        {тело метода}

type  method2(type1 var1, type1 var2) // публичный метод
        {тело метода}
virtual type  method3(type1 var1,..) // виртуальный метод
        {тело метода}
}
```

Наследование

```
 class Имя_Класса2: public_Имя_Класса1
 {
…
public: // публичные атрибуты/ методы

type  method2(type1 var1,..) override  // переопределение
 виртуального метода
        {тело метода}
 }
```

Концепции ООП

Фундаментальные концепции:

1. **Инкапсуляция;**
2. **Наследование;**
3. **Полиморфизм;**



Объявление класса (базового)

```
class Имя_Класса
{
   тело_класса
}
```

`тело_класса:`

1

```
Спецификатор_доступа :
private:   члены класса доступны только внутри этого класса
 (приватные члены класса) – по умолчанию.
protected: члены класса доступны только внутри этого класса и
 в потомках (защищенные члены класса)
public:    доступ к членам класса неограничен (открытые члены
 класса)
```

Объявление класса (базового)

```
class Имя_Класса
{
    тело_класса //-> Инкапсуляция: данные, методы и способы
                    доступа к ним
}
```

`тело_класса:`

1. Спецификатор доступа (private, protected, public):
2. Объявление членов класса (свойства-поля данных)
3. Объявление методов (функций-членов)

Объявление класса (базового)

```
class Имя_Класса
{
    тело_класса
};
```

```
тело_класса:
```

2

```
Объявление_членов_класса (атрибута):
 спецификатор_типа имя_члена1,…;
 спецификатор_типа имя_члена2 = значение_по_умолчанию;
```

Пример:

```
class Point
{
        double x=0,y=0; // приватные свойства x и y
}
```

Объявление класса (базового)

```
class Имя_Класса
{
   тело_класса
};
```

```
тело_класса:
```

2

```
Объявление членов класса (атрибута):
 спецификатор_типа имя_члена1,…;
 спецификатор_типа имя_члена2 = значение_по_умолчанию;
```

Пример:

```
class Point
{  public:
       double x=0,y=0; // открытые свойства x и y
}
```

Объявление класса (базового)

```
class Имя_Класса
{
    тело_класса
}
```

```
тело_класса:
```

3 `Объявление_методов`

3.1 полное определение метода (синтаксис функции)

```
префикс выходной_тип имя_метода(входные_параметры) постфикс
{
 тело функции
}
```

3.2 интерфейс(заголовок) метода-функции

```
префикс выходной_тип имя_метода(входные_параметры) постфикс
```

Объявление класса (базового)

```
...
```

```
тело_класса:
```

Объявление методов

(3.1) полное определение метода в теле класса:

Пример

```
class Point
{  public:
       double x,y;
       double rad() {return sqrt(x*x+y*y);}
}
```

Объявление класса (базового)

```
class Имя_Класса
{
    тело_класса
}
```

```
тело_класса:
```

Объявление_методов

3.2 интерфейс(заголовок) метода-функции:

```
префикс выходной_тип имя_метода(входные_параметры) постфикс
```

Реализация метода за пределами объявления класса

```
выходной_тип Имя_Класса::имя_метода(входные_параметры)
```

Объявление класса (базового)

```
...
```

```
тело_класса:
```

3.2

```
Объявление_методов
```

интерфейс(заголовок) метода-функции:

```
префикс выходной_тип имя_метода(входные_параметры) постфикс
```

Пример (объявление заголовка метода и его последующие опред.):

```
class Point
{  public:
       double x,y;
       double rad();
}
...
double Point::rad() { return sqrt(x*x+y*y); }
```

1. Объявление класса

Префикс метода:

пусто – (по умолчанию)
inline – встроенный вызов функции (метода)
virtual – виртуальный метод
friend – дружественная функция
static – статические методы

Постфикс метода:

пусто – (по умолчанию)
const – метод, не изменяющий значения свойств, может вызываться в экземплярах-константах.
override – перегруженный виртуальный метод
final – финализированный виртуальный метод

2. Экземпляр класса

Класс – это тип данных, экземпляр класса – это конкретный объект этого типа.



Объявление новой переменной как экземпляра класса

```
Имя_класса Имя_переменной
```

Доступ к свойствам и методам экземпляра

```
Имя_переменной.свойство_или_метод
```

Пример:

```
class Point{…}
  ..
int main()
{
    Point myPoint;
    myPoint.x = 1; myPoint.y = 2;
    double r = myPoint.rad();
}
```

2. Экземпляр класса

Объявление переменной – указателя на экземпляра класса

```
Имя_класса* Имя_указателя
```

Динамическое размещение (new) и удаление (delete) экземпляра из памяти

```
Имя_указателя = new Имя_класса; … delete Имя_указателя;
```

Доступ к свойствам и методам экземпляра

```
Имя_указателя->свойство_или_метод
```

Пример:

```
class Point{…};
  ...
int main()
{
    Point* myPoint=new Point;
    myPoint->x = 1; myPoint->y = 2;
    cout << myPoint->rad();
    delete myPoint;
}
```

Служебный формальный параметр this

this – это ссылка на экземпляр класса, для которого был вызван метод

```
выходной_тип имя_метода(входные_параметры)
{
   … this->свойство_или_метод …
}
```

Пример:

```
class Point
{  public:
       double x,y;
       double rad()
        {
        return sqrt(this->x * this->x + this->y * this->y);
        }
};
```

4. Конструкторы и деструктор

Конструктор– это особые метод класса, который вызывается при создании экземпляры класса.

Деструкторы – метод, выполняющий действия при ликвидации экземпляра класса

```
class Имя_Класса
{
    …
    Имя_Класса (входные_параметры) //конструктор
    ~Имя_Класса() // деструктор
};
```

```
class Point
{  public:
       double x,y;
       double rad();

       Point(double x0, double y0) {x=x0; y=y0;}
};
```

4. Конструкторы и деструктор

Конструктор– это особый метод класса, который вызывается при создании экземпляры класса.

Деструкторы – метод, выполняющий действия при ликвидации экземпляра класса

```
class Имя_Класса
{
    …
    Имя_Класса (входные_параметры) //конструктор
    ~Имя_Класса() // деструктор
};
```

```
class Point
{  public:
       double x,y;
       double rad();

       Point(double x, double y) {this->x=x; this -> y=y;}
};
```

4. Конструктор и деструктор

```
class Point
{  public:
       double* x,y;
       double rad();

       Point(double x, double y) // конструктор
       {
         this->x = new double(x);
         this->y = new double(y);
       }
     ~Point(){ delete x; delete y;} // деструктор
};

{
   Point myPoint = Point(1,2);
   Point* pPoint = new Point(1,2);
   …
   delete pPoint; // + вызван метод pPoint.~Point();
} // вызван деструктор myPoint.~Point();
```

Сеттер (setter) – метод для установки значения свойства

Геттер (getter) – метод для получения значения свойства

```
class Point
{
    double x,y; //приватные члены
  public:
    Point (x0,y0){x=x0; y=y0;} // конструктор
    void set_x(double x) { this->x=x;} // сеттер x
    double get_x() { return this->x;} // геттер x
};

…

Point p=Point(0,1);
// double x=p.x; // недоступно
double x=p.get_x(); // доступно
```

Перегруженные функции/методы – это функции с одинаковым именем, но разным набором параметров (интерфейсом)

```
выходной_тип_1 имя_функции (входные_параметры_1)
{
    тело_1
}

выходной_тип_2 имя_функции (входные_параметры_2)
{
    тело_2
}
```

Формальные параметры (типы данных или их очередность) или выходной тип данных должны отличаться, чтобы компилятор при трансляции мог отличить разные вызовы.

Пример:

```
class Point
{  public:
        double x,y;
        …
        double dist() // расстояние до центра коорд.
         {
         return sqrt(x*x+y*y);
         }

        double dist(Point b) // расстояние до точки b.
         {
           return sqrt((x-b.X)*(x-b.X)+(y-b.y)*(y-b.y));
         }
};

Point a,b;
… // инициализация
cout<<a.dist()<<endl; cout<<a.dist(b)<<endl;
```

Пример 2 :

```
 class Point
 {  public:
        double x,y;

        Point(double x0, double y0){ x=x0; y=y0;}
        Point(double xy0=0){x=xy0; y=xy0;}
}

Point a,b,o;

a=Point(1,2);
b=Point(1);
o=Point();
```

7. Конструктор копий

```
Point a, b;
a=Point(1,2);
b=a;  // что произойдет при присвоении?
```

```
class Имя_Класса
{
    …
    Имя_Класса (входные_параметры) //конструктор
    Имя_Класса (const Имя_Класса &Src) //конструктор копий

};
```

Пример 1:

```
class Point
{  public:
       double x,y;

       Point(double x0, double y0){ x=x0; y=y0;}
       Point(const Point &s){x=s.x; y=s.y;}
};
```

Пример 1:

```
class Point
{  public:
       double x,y;

       Point(double x0, double y0){ x=x0; y=y0;}
       Point(const Point &s){x=s.x; y=s.y;}
};


Point a = Point(1, 2);
Point b = a;

b.x=0; // a.x==1;
```

Пример 1:

```
class Point
{  public:
       double x,y;

       Point(double x0, double y0){ x=x0; y=y0;}
       Point(const Point &s){x=s.x; y=s.y;}
};


Point a = Point(1, 2); // a.x=1; a.y=2;
Point b = a; // b.x=1; b.y=2;

b.x=0; // a.x==1;
```

8. Конструкторы по умолчанию

По умолчанию создаются следующие конструкторы:

1) конструктор инициализации по умолчанию (пустой).

```
inline Имя_класса(){}
```

Не создается (!), если хотя бы один конструктор был объявлен.

2) конструктор копий

```
inline Имя_класса(const Имя_класса& copy)
```

Копируется побитово содержимое объекта copy

3) конструктор перемещений (копия по ссылке)

```
inline Имя_класса(Имя_класса&& copyptr)
```

8. Конструкторы по умолчанию

Пример :

```
class Point
{
public:
double x=0, y=0;
};


…

Point a;  // вызывается конструктор по умолчанию (пустой).
```

Конструкторы по умолчанию

Пример:

```
class Point
{
public:
double x=0, y=0;

Point(double x0, double y0) { x = x0; y = y0; }
};
…
Point a;  // !ошибка компиляции: не сущ. конструктора по умолчанию.
```

8. Конструкторы по умолчанию

Пример : явное объявление конструктора по умолчанию (вариант 1)

```
class Point
{
public:
double x=0, y=0;
Point() {};
Point(double x0, double y0) { x = x0; y = y0; }
};
…
Point a;  // неявная инициализация: a.x=0, a.y=0;
```

8. Конструкторы по умолчанию

Пример : явное объявление конструктора по умолчанию (вариант 2)

```
class Point
{
public:
double x=0, y=0;
Point() = default;
Point(double x0, double y0) { x = x0; y = y0; }
};
…
Point a;  // неявная инициализация: a.x=0, a.y=0;
```

8. Конструкторы по умолчанию

Пример : явное объявление конструктора по умолчанию (вариант 2)

```
class Point
{
public:
double x=0, y=0;
Point() = default;
Point(double x0, double y0) { x = x0; y = y0; }
};
…
Point a;  // неявная инициализация: a.x=0, a.y=0;
Point b=Point(1,2);

Point a2=b;  // использован неявный конструктор копий.
a=b;   // использован неявный оператор присвоения.
```

8. Конструкторы по умолчанию

Пример: конструктора копий по умолчанию

```
class Poligon
{
public:
Point* p=nullptr; // по умолчанию нулевая ссылка
Poligon() = default;
Poligon(Point* p) { копировать(p,this->p)}
Poligon(float* x, float* y) { инициализировать (this->p,x,y)}
};
…
Poligon p1= Poligon({1,2},{1,2});  //  (1,1),(2,2)
Poligon p2 = p1; // инициализатор копий по умолчанию p2.P=p1.P,
p1.p(1).x=0; // => p2.p(1).x == 0
```

8. Конструкторы по умолчанию

Пример: явное определение конструктора копий

```
class Poligon
{
public:
Point* p=nullptr; // по умолчанию нулевая ссылка
Poligon() = default;
Poligon(Point* p) { копировать(p,this.p);}
Poligon(float* x, float* y) { инициализировать this.p;}
Poligon(const Poligon& copy) { копировать(p, copy.p);}
};
…
Poligon p1= Poligon({1,2},{1,2});  //  (1,1),(2,2)
Poligon p2 = p1; // инициализатор копий: p2.p и p1.p – разные массивы
p1.p(1).x=0; //  p2.p(1).x == 1
```

8. Конструкторы по умолчанию: явное добавление и удаление

Служебное слово default: явное добавление конструктора по умолчанию

```
Конструктор = default
```

Служебное слово delete: явное удаление конструктора по умолчанию

```
Конструктор = delete
```

```
class Poligon
{
public:
Point* p=nullptr; // по умолчанию нулевая ссылка

Poligon() = default;
Poligon(const Poligon& copy) = delete; // конструктора копий уже нет
// т.е. запрещена инициализация вида: Poligon Poligon1= Poligon2
Poligon(Point* p) { копировать(p,this.p);}
};
```

Списки инициализации конструктора

```
class Имя_Класса
{
    Имя_Класса (входные_параметры): список_инициализации
    {
      тело
    }
};
```

```
class Имя_Класса
{
    Тип1 a;
    Тип2 b;
    Имя_Класса (Тип1 a0, Тип2 b0) : a(a0), b(b0)
    {};
};
```

Фактический порядок в списке инициализации = порядку объявления атрибутов! Лучше в списках не использовать атрибуты класса в «правой части» инициализации (напр., a(b))

9. Списки инициализации конструктора

Зачем это нужно?

```
class MyClassConstRef{
private:

 int i;
 const int ci0;
 int &ri;

public:

 MyClassConstRef(int i0, int ri0)
 {
   i=i0;
   ci0=i0;
   ri=ri0;    // ошибка
 }
};
```

Зачем это нужно?

```
class MyClassConstRef{
private:

 int i;
 const int ci0;
 int &ri;

public:

 MyClassConstRef(int i0, int ri0): ci0(i0), ri(ri0)
 {
   i=i0;
 }
};
```

9. Списки инициализации конструктора

1)

```
class Имя_Класса
{
    Имя_Класса (входные_параметры): список_инициализации
    {
      тело
    }
};
```

2)

```
class Имя_Класса
{
    Тип1 a;
    Тип2 b;
    Имя_Класса (Тип1 a0, Тип2 b0) : вызов_метода(a0,b0)
    {};
};
```

Пример :

```
class Point
{
public:
double x, y;

Point(double x0, double y0) { x = x0; y = y0; }
Point(double xy0) : Point(xy0, xy0) {};
Point() : Point(0.0, 0.0) { };
Point(const Point &s) { x = s.x; y = s.y; }

};
```

Здесь: `Point(double x0, double y0)` – это *делегирующий конструктор*

10. Статические члены класса

```
class Имя_Класса
{
    static <type> var // статическое поле
    static <type> func([<type> inp_var[,]]) // статический
                                            метод
};
```

Статические переменные (поля) и методы относятся непосредственно к классу, т.е. разделяются между всеми объектами класса.
Статические члены и методы предназначены для хранения и обработки информации, характерной для всех объектов класса.

10. Статические члены класса

```
class Point
{  public:
       double x,y;
       static int count=0;

       Point(double x0, double y0){ x=x0; y=y0; count++; }
       Point(const Point &s){x=s.x; y=s.y;}
       ~Point() {count--; }

       static void print_count() { cout >> count};
};
…

Point P0=Point(0,0);
P0.print_count(); // доступ через член класса

Point P1=Point(1,1);
Point::print_count(); // доступ через область видимости класса
```

Простое наследование: наследование от одного класса

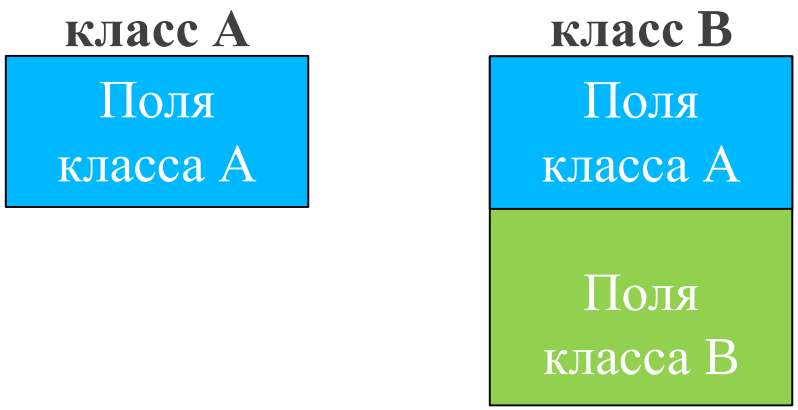
```
class класс_потомок: [модификатор_доступа] базовый_класс
 {
    …
 };
 модификатор_доступа:public, protected, private(по умолч)
```

Доступ при наследовании	доступ к полям базового класса		
	public	protected	private
public	public	protected	недоступны
protected	protected	protected	недоступны
private	private	private	недоступны

Открытое простое наследование

```
class A
  {
     …
  };

class B: public A   // открытое простое наследование
  {
     …
  };
```





13. Изменение доступа при наследовании

Оператор using изменяет доступ для неприватных членов.

```
модификатор_доступа:

  using имя_предка::поле;   // для неприватного поля предка
```

13. Изменение доступа при наследовании

```
class A
{ private:
      int a;
  protected:
     int  b,c;
     int func() { return c * 2; }
  public:
    int d;
};

class B: A
{
  protected:
      using A::b; // теперь b – защищенный
      using A::d; // теперь d – защищенный
   public:
     using A::c; // теперь c – публичный
     using A:: func; // теперь func– открытый метод
};
```

14. Переопределение методов при наследовании. Полиморфизм.

```
class C
{protected:
      int  b,c;
   public:
      C(int b, int c) { this->b = b; this->c = c; }
      int method1() { return b+c * 2; }

};

class D: public C
{
 public:
    D(int c) : C ::C(1, c) {}; // переопределение (перегрузка) конструктора
    int method1() { return c+b * 2; }
};

C v1 = C(1, 2);
D v2 = D(2);

int i1 = v1.method1();
int i2 = v2.method1();
```

14. Переопределение методов при наследовании.

```
class C
{protected:
      int  b,c;
   public:
      C(int b, int c) { this->b = b; this->c = c; }
      int method1() { return b+c * 2; }
      int method2() { return method1()*3; } !ЛОГИЧЕСКАЯ ОШИБКА
};

class D: public C
{
 public:
    D(int c) : C ::C(1, c) {}; // перегрузка конструктора
    int method1() { return c+b * 2; }
};

C v1 = C(1, 2);
D v2 = D(2);

int i1 = v1.method1(),   j1=v1.method2();
int i2 = v2.method1(),   j2=v2.method2();
```

14. Переопределение методов при наследовании.

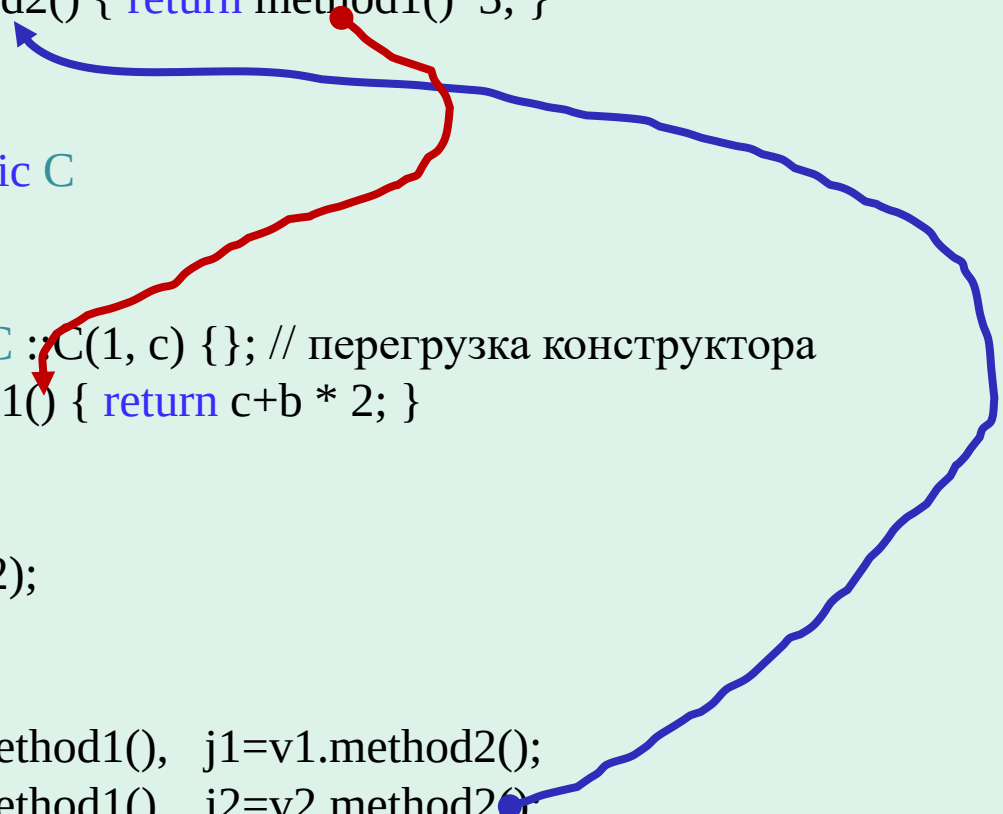
```
class C
{protected:
      int  b,c;
   public:
      C(int b, int c) { this->b = b; this->c = c; }
      int method1() { return b+c * 2; }
      int method2() { return method1()*3; }
};

class D: public C
{
 public:
    D(int c) : C ::C(1, c) {}; // перегрузка конструктора
    int method1() { return c+b * 2; }
};

C v1 = C(1, 2);
D v2 = D(2);

int i1 = v1.method1(),   j1=v1.method2();
int i2 = v2.method1(),   j2=v2.method2();
```

Ожидаем



14. Переопределение методов при наследовании.

```
class C
{protected:
     int  b,c;
  public:
     C(int b, int c) { this->b = b; this->c = c; }
     int method1() { return b+c * 2; }
     int method2() { return method1()*3; }
};

class D: public C
{
 public:
    D(int c) : C ::C(1, c) {}; // перегрузка конструктора
    int method1() { return c+b * 2; }
};

C v1 = C(1, 2);
D v2 = D(2);

int i1 = v1.method1(),   j1=v1.method2();
int i2 = v2.method1(),   j2=v2.method2();      // j1==j2
```

А на самом деле

Не удовлетворяет требованию полиморфизма

15. Связывание

Связывание – это сопоставление имени с адресом по которому хранятся данные либо инструкции (машинный код процедуры).

Т.е. понятие связывание относится как к переменным, так и к функциям.

Классификация:

1) Раннее (или статическое) связывание: осуществляется до запуска программы на этапе ее трансляции.

2) Позднее (или динамическое) связывание: осуществляется во время выполнения программы

Примеры:

```
 int   a,b; //переменная
 int   *p; //ссылка

 a = 1; // раннее связывание


 …
 p = &a;
*p = 2; // позднее связывание

 p = &b;
*p = 2*a; // позднее связывание
```

15. Связывание функций

Для обычных методов класса, а также обычных функций, для которых программист указал в тексте программы имя связывание раннее.

```
float   a; //переменная
a  = cos(2.0); // раннее связывание с cos(x)
```

Использование указателей на функцию – пример позднего связывания.

синтаксис указателей на функцию: (имя параметров можно опустить)

Возвращаемый_тип (*имя_уазателя) (тип [имя_параметра],…)

```
float (*f)(float); //объявление указателя f на
функцию с прототипом  float имя(float)
f  = &cos;
float a = f(2.0)  // позднее связывание с cos(x)
```

Виртуальные функции – механизм реализации позднего связывания и, соответственно, полиморфизма в ООП C++ .

Спецификатор virtual

```
virtual тип имя_метода (тип имя_параметра,…)
```

При перегрузке виртуального метода в классе потомка, необходимо сохранить прототип метода исходного класса, т.е. имя функции, число и типы входных и выходных параметров.



16. Виртуальные методы и позднее связывание в ООП. Полиморфизм.

```
class C
{protected:
      int  b,c;
   public:
      C(int b, int c) { this->b = b; this->c = c; }
     virtual int method1() { return b + 2*c; }
      int method2() { return method1()*3; }
};

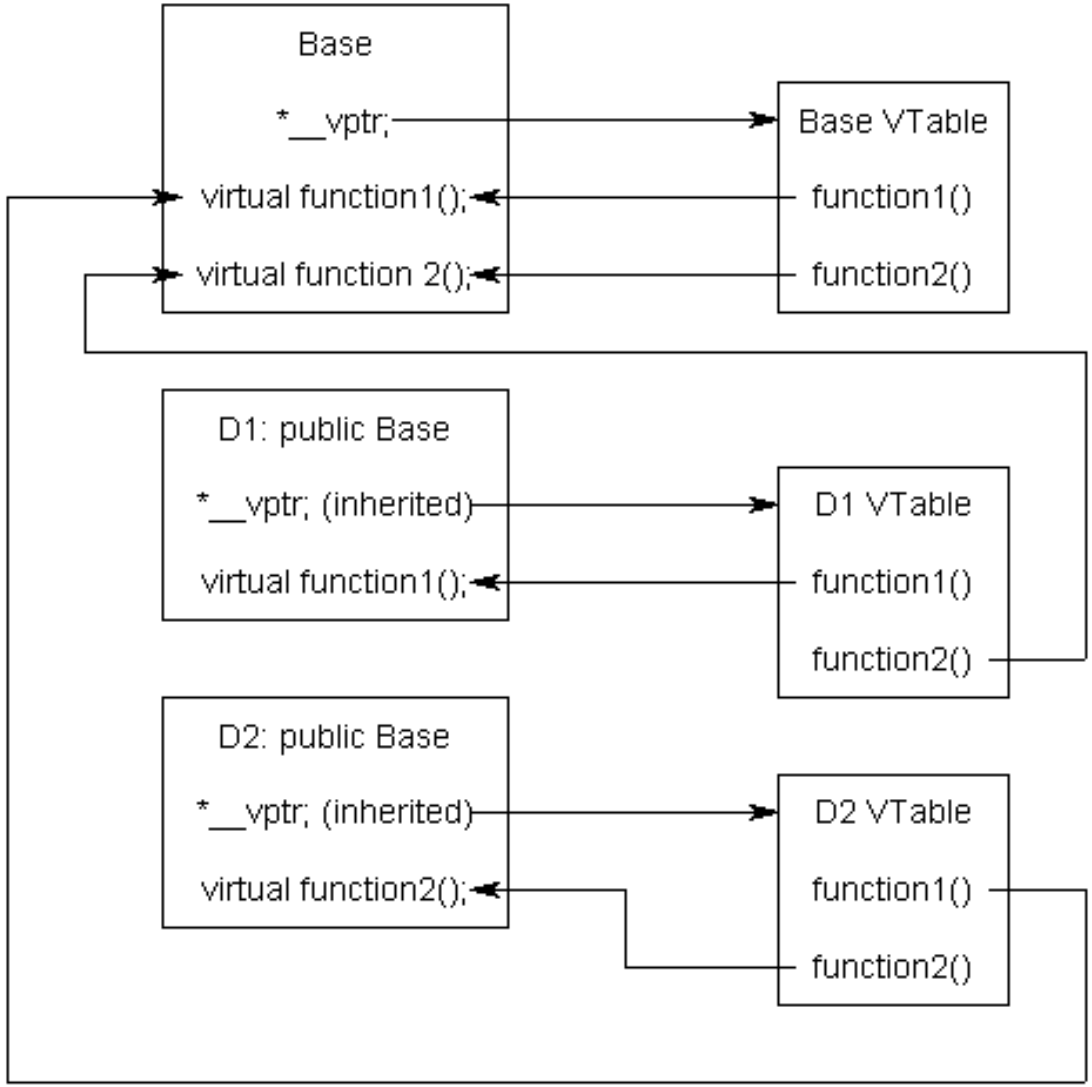
class D: public C
{
 public:
    D(int b, int c) : C ::C(1, c) {};
     int method1() override  // использование спецификатора override не обязательно, но это гарантия проверки компилятором на ошибки.
          { return c+b * 2; }  // переопределение виртуального метода
};

C v1 = C(1, 2);
D v2 = D(2);
int i1 = v1.method1(),   j1=v1.method2();
int i2 = v2.method1(),   j2=v2.method2();
```

16. Таблица виртуальных методов

C++ для реализации виртуальных функций использует специальную форму позднего связывания – через механизм виртуальных таблиц. **Виртуальная таблица** – это таблица, в которой хранятся адреса всех виртуальных функций. Используется для осуществления поиска функций и разрешения проблемы вызовов функций в режиме динамического связывания. Адрес таблицы хранится в скрытых параметрах класса.

16. Таблица виртуальных методов





16. Виртуальные методы и позднее связывание в ООП

Правила использования виртуальных функций

1) Виртуальная функция наследуется
2) Статические методы и конструкторы не могут быть виртуальными
3) Деструкторы могут быть виртуальными.
4) Перегрузка операторов м.б. виртуальной.

16. Чистые виртуальные функции и абстрактные классы

Виртуальная функция без реализации называется чистой, объявляется следующим образом:

```
virtual  тип  имя_метода (параметры) = 0
```

Класс, содержащий хотя бы один чистый метод, называется абстрактными.

Создавать объекты абстрактного класса запрещено.

Потомки абстрактного класса должны перегрузить (вернее определить) тело чистой функции.

Правило наследования конструкторов и деструкторов

- ✓ Конструкторы не наследуются, поэтому производный класс должен иметь собственные конструкторы.
- ✓ Если в конструкторе производного класса явный вызов конструктора базового класса отсутствует, автоматически вызывается конструктор базового класса по умолчанию (то есть тот, который можно вызвать без параметров).
- ✓ Для иерархии, состоящей из нескольких уровней, конструкторы базовых классов вызываются начиная с самого верхнего уровня. После этого выполняются конструкторы тех элементов класса, которые являются объектами, в порядке их объявления в классе, а затем исполняется конструктор класса.
- ✓ В случае нескольких базовых классов их конструкторы вызываются в порядке объявления.
- ✓ Если конструктор базового класса требует указания параметров, он должен быть явным образом вызван в конструкторе производного класса в списке инициализации

Правило наследования конструкторов и деструкторов

- ✓ Деструкторы не наследуются, и если программист не описал в производном классе деструктор, он формируется по умолчанию и вызывает деструк-торы всех базовых классов.
- ✓ В отличие от конструкторов, в деструкторе производного класса не требуется явно вызывать деструкторы базовых классов, поскольку это будет сделано автоматически.
- ✓ Для иерархии классов, состоящей из нескольких уровней, деструкторы вызываются в порядке, строго обратном вызову конструкторов: сначала вызывается деструктор класса, затем – деструкторы элементов класса, а потом деструктор базового класса.

Вызов методов базового класса

Для доступа к методам базовых классов используется оператор :: расширения области видимости

```
class C
{protected:
      int  b,c;
   public:
      C(int b, int c) { this->b = b; this->c = c; }
     virtual int method1() { return b + 2*c; }
      int method2() { return method1()*3; }
};

class D: public C
{
 public:
    D(int b, int c) : C::C(1, c) {};
     int method1() override
          { return C::method1()+b-c; }  // переопределение виртуального метода
};
```

Перегруженная операция – это функция, определенная для некоторого набора входных параметров, вызываемая после применения к фактическим параметров указанной операции. Может быть как член класса, так и отдельная функция.

```
выходной_тип operator@ (входные_параметры)
{
    тело_1
}

@ - символ операции.
```

Перегрузка операций и функций – это одно из проявлений полиморфизма (параметрического)

Операторы, которые могут быть перегружены					
+	-	*	/	%	^
&	|	~	!	,	=
<	>	<=	>=	++	--
<<	>>	==	!=	&&	||
+=	-=	/=	%=	^=	&=
|=	*=	<<=	>>=	[]	()
->	->*	new	new []	delete	delete []
Операторы, которые не могут быть перегружены					
::	.*	.	?:		

Не рекомендуется перегружать: , && || -> ->* new delete

11. Перегрузка операций

```
class T
{
public:
int  a=0;
T& operator+(int b) { a=a+b; return *this;}
};
…

T t;
t=t+1;  // t.a=t.a+1;  == 1;

t=2+t;  // ошибка!
```

```
class T
{
public:
int  a=0;
T& operator+(int b) { a=a+b; return *this;}
};

T& operator +(int b, T& obj) { return obj+b; }

T t;
t=t+1;  // t.a=t.a+1;  t.a == 1;

t=2+t;  // t.a=t.a+2; t.a == 3;
```

```
class T2
{
int  a=0; // приватный член
public:
…
};

int operator +(int b, T2& obj) { return obj.a+b; } // ошибка!
int operator +(T2& obj, int b) { return obj.a+b; } // ошибка!

T2 t;
int i=t+1;
```

11 Дружественные функции

friend – служебная слово, дающее возможность доступа к любым элементам данного класса из функции (или другого класса). Дружественная функция – формально не член класса

```
class Имя_Класса
 {
  …
 friend выходной_тип функция(входные_параметры,
Имя_Класса obj)
     {
      тело_1
     }
 …
 };
```



11. Перегрузка операций: дружественные функции

```
class T
{
int  a=0; // приватный член
public:
friend int operator +(int b, T& obj) { return obj.a+b; }
friend int operator +(T& obj, int b) { return obj.a+b; }
};


T t;
int i=t+1;
```

11 Часто используемая форма перегрузки операторов

Операция	Рекомендуемая форма перегрузки
Все унарные операции	Метод класса
= [] () –> –>*	Обязательно метод класса
+= –= *= /= %= &= |= ^= <<= >>=	Метод класса
Остальные бинарные операции	Внешняя функция-друг
Преобразование типов	Обязательно метод класса

11. Перегрузка операций ввода-вывода

```
class T
{
int  a=0; // приватный член
public:
 friend  ostream& operator<<(ostream& ios, T& obj)
    { return ios << obj.a; }
  friend istream& operator>>(istream& ios, T& obj)
   {ios >> obj.a; return ios; }
};



T t;
cin>> t;
cout<<t;
```

11 Перегрузка операторов присваивания

```
class T
{
int  a=0; // приватный член
public:
    T& operator=(int b) { this->a=b; return *this; }
     // оператор присваивания по умолчанию
     // T& operator=(const T &b)
};

T t1,t2;
 t1=2;
 t2=t1;
 t1=3;  // t2.a==2; t1.a==3;
```

11. Перегрузка операторов присваивания

```
class T
{
int  a=0; // приватный член
public:
    T& operator=(int b) { this->a=b; return *this; }
     // оператор присваивания по умолчанию указан явно
    T& operator=(const T &b)  = default;
};

T t1,t2;
 t1=2;
 t2=t1;
 t1=3;  // t2.a==2; t1.a==3;
```

11 Перегрузка операторов присваивания

```
class T
{
int  a=0; // приватный член
public:
    T& operator= (int b) { this->a=b; return *this; }
     // оператор присваивания по умолчанию удален
    T& operator= (const T &b)  = delete;
};

T t1,t2;
 t1=2;
 t2=t1; !ошибка: ссылка на удаленную функцию.
 t1=3;
```

11. Перегрузка операторов присваивания

```
class Poligon
{
void copy_points(Point* cp) {…};
public:
Point* p=nullptr; // по умолчанию нулевая ссылка

Poligon() = default;
Poligon(const Poligon& copy) {copy_points(copy.p);}
Poligon(Point* p) {copy_points(p);}

 Poligon& operator= (const Poligon &obj)
{copy_points(obj.p); return *this;}
};
```

Операция присваивания не наследуется

Допускается неявное преобразование типов

```
operator тип() const
```

Явное преобразование типов

```
explicit operator тип() const
```

Требования: является функцией-членом класса, тип возвращаемого значения не указывается явно, т.к. совпадает с типом данного класса.

11. Операторы преобразования типов

```
class T
{
int  a=0; // приватный член
public:
    T& operator=(const T &obj)  = default;
    T& operator=(const int b) {a=b};

   // оператор преобразования типов
   operator int() const {return this->a;}
};

 T t1,t2;
 t1=2;
 t2=2+t1;  // ⇔ t2=t1+2;  =>  t2.a==4;
```

11 Операторы преобразования типов

```
class T
{
int  a=0; // приватный член
public:
   T& operator=(const T &obj)  = default;
    T& operator=(const int b) {a=b};

  // оператор преобразования типов
   explicit operator int() const {return this->a;}
};

 T t1,t2;
 t1=2;
 t2=2+t1;  //  ошибка!
```

11. Операторы преобразования типов

```
class T
{
int  a=0; // приватный член
public:
    T& operator=(const T &obj)  = default;
    T& operator=(const int b) {a=b};

    // оператор явного преобразования типов
   explicit operator int() const {return this->a;}
};

 T t1,t2;
 t1=2;
 t2=2+(int)t1;  //  t2.a==4;     t2=2+int (t1);
```

Ещё раз об операторах в C++

Приоритет	Оператор	Описание
1	`::`	Разрешение области видимости
2	`a++ a--`	Суффиксный/постфиксный инкремент и декремент
	тип`()` *тип*`{}`	Функциональный оператор приведения типов
	`a()`	Вызов функции
	`a[]`	Индексация
	`. ->`	Доступ к элементу
3	`++a --a`	Префиксный инкремент и декремент
	`+a -a`	Унарные плюс и минус
	`! ~`	Логическое НЕ и побитовое НЕ
	(*тип*)	Приведение типов в стиле C
	`*a`	Косвенное обращение (разыменование)
	`&a`	Взятие адреса
	`sizeof`	Размер в байтах
	`co_await`	Выражение await (C++20)
	`new new[]`	Динамическое распределение памяти
	`delete delete[]`	Динамическое освобождение памяти
4	`.* ->*`	Указатель на элемент
5	`a*b a/b a%b`	Умножение, деление и остаток от деления
6	`a+b a-b`	Сложение и вычитание
7	`<< >>`	Побитовый сдвиг влево и сдвиг вправо
8	`<=>`	Оператор трёхстороннего сравнения (C++20)
9	`< <= > >=`	Для операторов отношения < и ≤ и > и ≥ соответственно
10	`== !=`	Операторы равенства = и ≠ соответственно
11	`&`	Побитовое И
12	`^`	Побитовый XOR (исключающее или)
13	`|`	Побитовое ИЛИ (включающее или)
14	`&&`	Логическое И
15	`||`	Логическое ИЛИ
16	`a?b:c`	Тернарный условный оператор
	`throw`	Оператор throw
	`co_yield`	Выражение yield (C++20)
	`=`	Прямое присваивание
	`+= -=`	Присваивание с сложением и вычитанием
	`*= /= %=`	Присваивание с умножением, делением и остатком
	`<<= >>=`	Присваивание с побитовым сдвигом влево и вправо
	`&= ^= |=`	Присваивание с побитовым И, XOR и ИЛИ
17	`,`	Запятая

12. Вложенные и дружественные классы

```
class A
 {
   модификатор_доступа:// {private},public, protected
   class B
     {
      тип method(параметры);
     };

    B b; // поле b типа B
    …
 };

 …

 тип A::B::method(параметры)
 { … }
```



12. Вложенные и дружественные классы

```
class B
 {
    …
 };

class A
 {
   friend class B; // объявление B дружественным
 классом: все поля A доступны B;
 };
```



Объявление класса дружественным позволяет снимать ограничения доступа.

12. Вложенные и дружественные классы

```
class TB
{ private:
      int b;
      friend class TA;
};

class TA
{
private:
 int a;
public:
      int methodA(TB oB) { return oB.b * 2; }

};
```

12. Вложенные и дружественные классы

```
class TB
{ private:
      int b;
  public:
      int methodB(TA oA) { return oA.a * 3; } !ошибка
 friend class TA;
};

class TA
{
private:
 int a;
public:
      int methodA(TB oB) { return oB.b * 2; }
};
```

Обобщённое программирование (generic programming) - методология программирования, заключающаяся едином описании структур данных и алгоритмов, которое при этом можно применять к различным типам входных данных.

В обобщённом программировании вместо описания отдельных типов описывается семейство типов, имеющих общий интерфейс и семантическое поведение (semantic behavior). Набор требований, описывающий интерфейс и семантическое поведение, называется концепцией (англ. concept). Таким образом, написанный в обобщённом стиле алгоритм может применяться для любых типов, удовлетворяющих его своими концепциями. Такая возможность называется полиморфизмом.

Впервые были реализовано в языках Клу и Ада в виде дженериков (обобщенных функций) в 1970х.

В языке C++ обобщённое программирование осуществляется в основном на понятии «шаблона» (ключевое слово template).

Большой вклад в появление развитых средств обобщённого программирования в C++ внёс Александр Степанов (русско-американский учёный в области информатики и вычислительной техники, разработчик STL (стандартной билиотеки шаблоннов) C++. В 1977 году эмигрировал в США.).

```
template <параметры>
 заголовок
 {
    тело
 }
```

Параметры:

1) `typename имя_типа` – параметр-тип (вместо `typename` допустим спецификатор `class`)
2) `имя_типа имя_параметра` – параметр указанного типа

Конструкция **template <параметры>** помещается перед объявлением функции либо класса. Параметры **template** используются в реализации функции (класса). При такой реализации функцию (класс) называют шаблоном.

Использование шаблона

Вызов шаблона-функции

```
имя_функции<параметры шаблона>(параметры функции)
```

Если интерпретация однозначная, то некоторые параметры шаблона можно опускать (см. пример далее)

Задание типа переменной через шаблон-класс

```
имя_класса<параметры шаблона>  имя_переменной
```

Пример шаблона-функции

```
template <typename T>
T Min(T  a, T b)
{
  if(a<=b)
     return a;
  else
    return b;
};

int main()
{
  int a=1, b=2;
  cout << Min(a,b);

  float u=3.0, v=2.0;
  cout << Min(u,v);
  …
}
```

Пример шаблона-функции

```
template <typename T,  int N>
T Min(T  ar[N])
{
   T min=ar[1];

    for (int i=1; i<N; i++)
    {
        if(min>ar[i]) min=ar[i];
     }
     return min;
};

int main()
{
    cout << Min<int,4>([11,-2,3,5]);
 }
```

Пример шаблона-класса

```
template <typename T>
class A
{
       T  v; // поле v типа T;
  public:
       A(T in_v) { v=in_v; } // конструктор;
      T inc() {return ++v;}
      T dec() {return --v;}
};

int main()
{
   A<int> v= A<int>(1);
   v.inc();
  …
 }
```

Пример шаблона-класса: определение метода вне тела класса

```
template <typename T>
class A
{
      T  v; // поле v типа T;
  public:
      A(T in_v) { v=in_v; } // конструктор;
      T inc()
      T dec() {return --v;}
};

template <typename T>
T A<T>::inc() {return ++v;}
```

17. Параметры шаблона по умолчанию

template <параметр=значение>

```
template <typename T = int>
class A
{
        T  v; // поле v типа T;
   public:
        A(T in_v) { v=in_v; } // конструктор;
       T inc() {return ++v;}
       T dec() {return --v;}
};

int main()
{
   A i= A (1);
   A<double> f= A<double> (2.73);

}
```

Литература

1. Липпман, С.Б. Язык программирования C++. Базовый курс, -- М.: Вильямс, 2014 – 1120 с.
2. Сиддхартха Рао. Освой самостоятельно C++. – М: Вильямс., 2013. – с 651.
3. Бьерн Страуструп. Программирование: принципы и практика использования C++ – М: Вильямс, 2011. – 1248 с.: